MS2 Summary — Padel Predictor

Noah Rodriguez · github.com/Noah-Rod/padel-predictor

1 What I achieved in this milestone

- **Live, scheduled ingestion from padelapi.org.** `.github/workflows/feature-pipeline.yml` runs daily at 04:15 UTC (seven days back, seven days of fixtures) and backfills a date range on demand in resumable 30-day chunks. `HttpMatchSource` (`features/sources.py`) follows the provider's OpenAPI spec: tournaments at the proposal's levels, then each one's main draw, paged and paced under the plan's rate limit, with `Retry-After` honoured. Walkovers, byes, undrawn pairs and results the free plan hides are dropped, not turned into fixtures. Every run rebuilds all features from scratch, because Elo and form depend on match order.
- **Hopsworks and W&B, running against the hosted services.** Raw matches, rankings, features and predictions are Hopsworks feature groups (primary key, event time, upsert); every training run is a W&B run, and the promoted model carries the `champion` alias that inference loads. Offline backends with the same contract keep the tests and a fresh clone running without an account.
- **One feature definition, leakage-tested.** `common/features.py` is used by training and inference. New in MS2: rankings joined on publication time (E1), the provider's Elo stored as a baseline beside every row (A2), and same-day rounds replayed in draw order.
- **Reproducible.** Docker image, `uv.lock`, 117 unit tests and lint on Python 3.11 and 3.12 in CI; Hopsworks is an optional extra, so a clone also installs on Windows.

Look at first: `parse_records` in `features/sources.py`, `attach_rankings` in `common/features.py` with the leakage tests in `tests/test_features.py`, and the step summary of the feature workflow, which ends with `padel data-report` read back from the store.

What it cost. The first live runs exposed four faults the stand-in tests could not: an empty `HOPSWORKS_HOST` broke the login; Hopsworks' Kafka upload needs a keystore `pyjks` cannot decode, so the groups are now DELTA, written directly (`fea38ab`); `| tee` made failed runs look green, fixed with `pipefail` (`a3ecd9d`); and FIP matches without a start time share midnight, so a quarter-final could be replayed before that day's round of 16 and leak into it (`4827680`).

Results, free and paid plan. I kept both. The free plan reaches six months back and shows only the current ranking, so no test match has a snapshot published before it: its provider-Elo bar is a constant 0.5. Each W&B run is tagged with its plan.

	Free plan (W&B run `1tq3lkm7`)	**Paid plan**
Labelled matches	1,674 (2026-05-08 to 11-01)	*rerun pending*
Test set	330 matches from 2026-10-12	
Model: log loss / accuracy	0.559 / 68.5 %	
In-house Elo	0.632 / 61.8 %	
Provider Elo (coverage)	0.693 / 54.8 % (0 %: no snapshot)	
Majority class	0.692 / 54.8 %	

2 Response to MS1 reviews

Labels as in the editorial summary. Reviewer A's suggestion is A2, reviewer B's suggestion is B1. Code: commits `a2fb2a5`, `9a3913f` and `9efda66` (live source); documentation: `0b5feb8`.

- **A2 · Baseline is not measured on my test set — Partly** (also reviewer A's suggestion). The provider's Elo comes from the ranking snapshot published strictly before each match, is stored as `provider_elo_prob` beside each feature row (never as a model input) and is scored on exactly the model's test rows. A test match without a snapshot gets 0.5 instead of being dropped, so the bar is not set on a friendlier subset, and the coverage is reported. *Now measured* on the live free-plan data (table above): coverage 0 %, so the bar is 0.5 and not yet meaningful. *Not yet:* the paid-plan number, which needs the ranking history, and registering the provider's Elo as the first champion (MS3). *Where:* `training/dataset.py` (`provider_elo_probabilities`), `features/report.py`.
- **E1 · Weekly snapshots can leak same-week results — Changed.** Rankings are now their own raw dataset with a `published_at` column. Each player gets the latest snapshot published *strictly before* the scheduled start (`merge_asof` without exact matches), never the one matching the label date. Backfilled snapshots without a publication time count as published seven days after their label date, so a snapshot labelled with a tournament week is never visible inside that week; re-reading a snapshot keeps the first time it was seen. Tests: a snapshot published after the week, one published exactly at the start, and rewriting every later snapshot leaves the match's

row unchanged. *Where:* `common/features.py` (`attach_rankings`), `common/schemas.py`, `tests/test_features.py`.

- **A1 · No minimum margin on ~2,000 test matches — Changed.** A paired bootstrap (2,000 resamples) of the per-match log-loss difference, model minus provider Elo. The criterion is met only if the whole 95 % interval lies below zero; it is logged as `vs_provider_elo_ci_low/high` and the tag `beats_provider_elo`. I chose an interval over a fixed margin, because a margin would be a guess about a test-set size I do not know yet. A test checks that a 0.001 edge on 2,000 matches does not count as a win. *Where:* `training/dataset.py` (`bootstrap_log_loss_difference`), `config/settings.yaml`.
- **E2 · Paid data tier: real volume and a fallback — Partly.** Real counts from the store (`padel data-report`, free plan): 1,674 labelled main-draw matches since May 2026 — FIP Gold 526, P1 516, Major 258, P2 212, FIP Platinum 162 — and 188 requests for the six-month backfill. The fallback is not hypothetical any more: it is what ran. `PADEL_API_PLAN=free` clamps the backfill to 183 days and stores only the current ranking, published at fetch time, so no historical match gets points it could not have had. Without any key, the `sample` source runs everything. *Not yet:* the paid-plan counts back to 2023 (rerun pending). *Where:* `features/report.py`, `features/sources.py`, README "Going live".
- **B1 · How player values become team features — Changed.** Elo: mean of the two players. Ranking points: sum, from the snapshot. Form: mean of each player's win rate over their last 5 matches with any partner, 0.5 without history. Rest: the smaller of the two players' gaps, capped at 60 days, 14 if never seen. Partnership: matches this exact pair has played together. A new partnership starts at 0 but keeps each player's Elo, form and rest; a test covers this. *Where:* README "What the model sees", `common/features.py` docstring, `tests/test_features.py`.
- **E3 · Report what partnership and rest add — Not yet.** Ablations belong to the training pipeline (MS3). The feature set stays at six, and the provider's Elo was taken out of the features so it stays a clean bar. In MS3, partnership, form and rest are each dropped in turn and every run's interval against the provider's Elo is compared in W&B. *Where:* planned in `training/pipeline.py`.
- **E4 · Log which model served which day — Changed.** Every stored prediction row carries `model_name`, `model_version`, `model_run_id` and `predicted_at` in the `predictions` feature group (primary key `match_id`, `run_date`), so the last pre-match prediction per match is the latest `predicted_at` before `played_at`. Scoring it live is MS4 monitoring. *Where:* `inference/predict.py` (`predict_frame`), `tests/test_inference.py`.